



VIT[®]
BHOPAL

CSE PROJECT REPORT

TEXT BASED PYTHON POKEMON GAME

NAME - ABHISHREY SHARMA

REGISTRATION NO. - 25BCE10150

BRANCH - BTECH CSE CORE

FACULTY – DR. SASHMITA PADHY

SLOT- E-11, E-12, C-14

(CSE1021)

INTRODUCTION

This project is a terminal-based Pokémon adventure game coded in Python. It allows players to name their trainer, choose a Pokémon starter, battle wild Pokémon, and save/load game progress—all through the command line.

PROBLEM STATEMENT

Create an interactive, text-based Pokémon game that simulates basic battle mechanics and enables persistent game progress, providing a simple and enjoyable experience while demonstrating Python programming concepts.

FUNCTIONAL REQUIREMENTS

- . Input trainer name at start
- . Choose a starter Pokémon from a list (Charmander, Bulbasaur, Squirtle, Pikachu)
- . Show Pokémon stats and health status
- . Initiate wild battles with randomly generated Pokémon
- . Display round-by-round combat including health changes
 - . Allow saving game state and reloading progress
- . Clear terminal for a clean UI display

NON-FUNCTIONAL REQUIREMENTS

- . Game should run in any standard Python environment (no external dependencies other than standard library)
- . User-friendly and robust input handling
- . Fast response and minimal loading time
- . Simple file-based storage for game progress

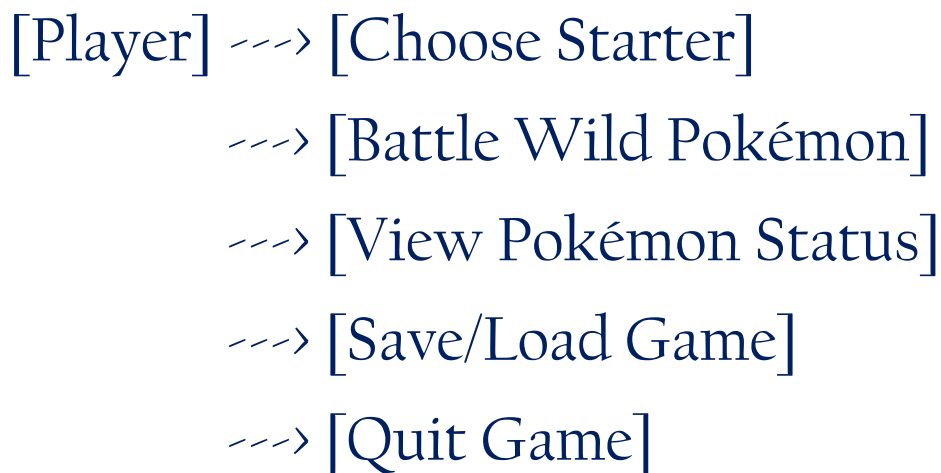
SYSTEM ARCHITECTURE

Components:

- . Player input and output handled via console
- . Main control loop for navigation
- . Python file I/O (JSON) for save/load
- . In-memory game state tracking (player and Pokémon objects)

DESIGN DIAGRAMS

a. Use Case Diagram



b. Workflow Diagram

Start -> Name Trainer -> Choose Starter -> Main Menu

v

v

(Wild Battle) (View Status/Save/Quit)

v

repeat

c. Sequence Diagram (Battle)

text Player initiates battle -> Battle starts -> Wild Pokémon chosen

|-> Round begins: attacker decided, damage calculated, HP updated

|-> Status printed -> Repeat until one Pokémon falls

|-> Outcome displayed

d. ER Diagram (Storage)

Not complex (only one entity stored):

text

```
Trainer {  
    name: string  
    pokemon: {  
        n: name  
        mh: max_hp  
        ch: curr_hp  
        a: attack  
        d: defense    }  
}
```

Design Decision and Rationale

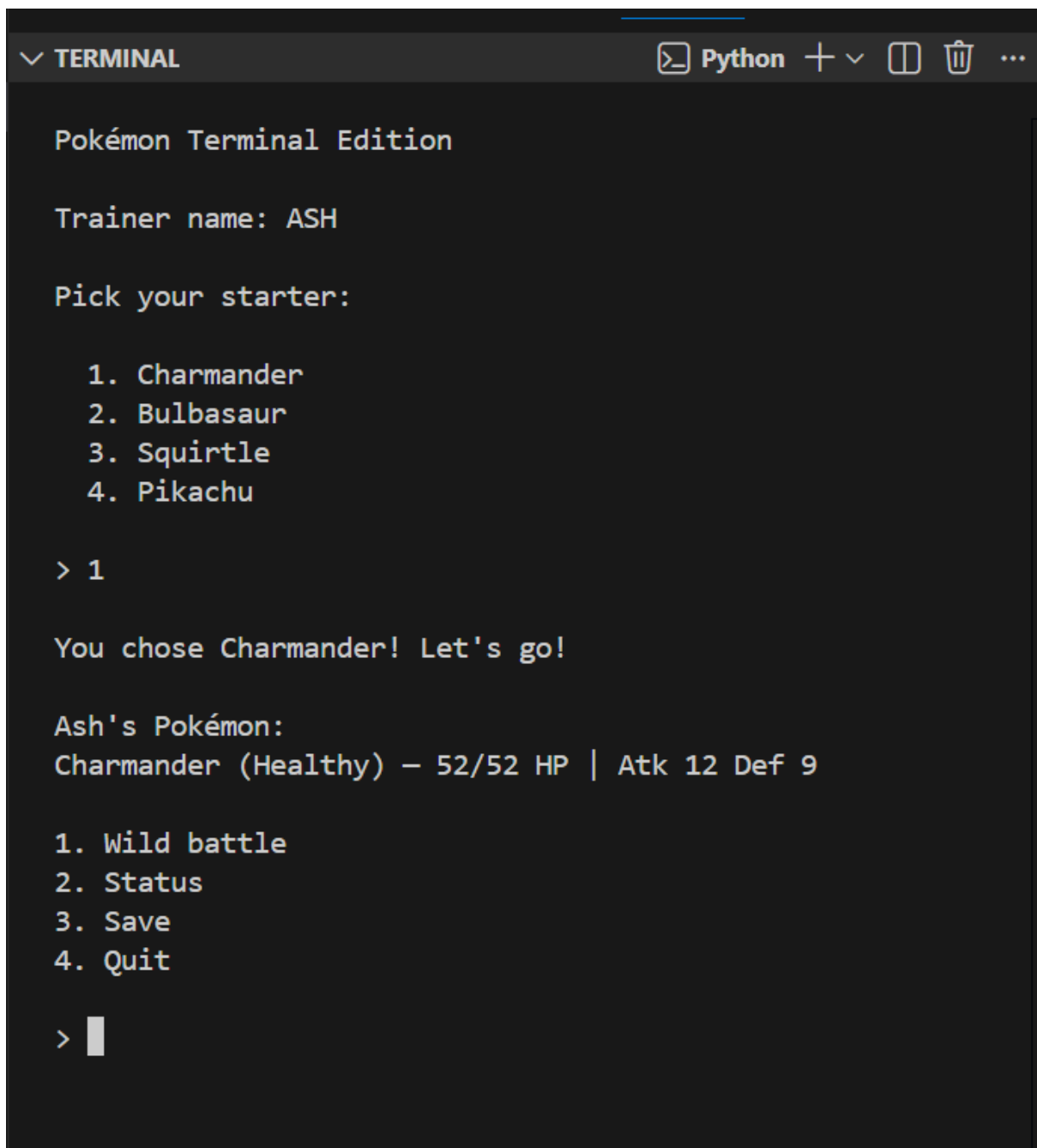
- Used simple object-oriented design for extensibility
- Chose JSON for persistent game state due to native Python support and readability
- Input handling with strong validation for a smooth user experience

Implementation Details

- Two main classes: Trainer and Pokemon
- All operations handled through console prompts and printed output
- Combat implemented with randomized attack order and damage within bounds
- Save/load feature reads and writes trainer and Pokémon info to a JSON file

- Terminal is auto-cleared (cross-platform)
for UI clarity

SCREENSHOTS / RESULTS



```
✓ TERMINAL Python + ▢ 🗑️ ...

Pokémon Terminal Edition

Trainer name: ASH

Pick your starter:

  1. Charmander
  2. Bulbasaur
  3. Squirtle
  4. Pikachu

> 1

You chose Charmander! Let's go!

Ash's Pokémon:
Charmander (Healthy) — 52/52 HP | Atk 12 Def 9

  1. Wild battle
  2. Status
  3. Save
  4. Quit

> █
```

> 1

A wild Rattata appeared!

Go Charmander!

Round 1

Charmander hits for 10!

Charmander (Healthy) – 52/52 HP | Atk 12 Def 9

Wild Rattata: 30/40 HP

Round 2

Charmander hits for 5!

Charmander (Healthy) – 52/52 HP | Atk 12 Def 9

Wild Rattata: 25/40 HP

Round 3

Wild Rattata hits for 4!

Charmander (Healthy) – 48/52 HP | Atk 12 Def 9

Wild Rattata: 25/40 HP

Round 4

Wild Rattata hits for 8!

Charmander (Healthy) – 40/52 HP | Atk 12 Def 9

Wild Rattata: 25/40 HP

Round 5

Charmander hits for 9!

Charmander (Healthy) – 40/52 HP | Atk 12 Def 9

Wild Rattata: 16/40 HP

```
Ash's Pokémon:  
Charmander (Healthy) — 52/52 HP | Atk 12 Def 9
```

1. Wild battle
2. Status
3. Save
4. Quit

```
> 3
```

```
saved
```

```
Ash's Pokémon:  
Charmander (Healthy) — 52/52 HP | Atk 12 Def 9
```

1. Wild battle
2. Status
3. Save
4. Quit

```
>
```



```
> 4
```

```
bye
```

- Initial trainer naming and starter selection
- Main menu options
- Battle output (rounds, health, win/loss)
- Save confirmation
- Reload with player and Pokémon restored

Testing Approach

- Manual testing for each feature: name entry, starter choice, battle loop, save/load, menu navigation
- Input validation tested for edge cases (invalid menu options, input types)
- Verified save/load by exiting and relaunching game

Challenges Faced

- Handling game state persistency across sessions
- Designing robust input handling for a seamless experience
- Balancing simplicity with enough game logic to make play engaging

Learnings & Key Takeaways

- Gained practical experience with Python OOP, file I/O, and basic game logic
- Understood importance of user interactivity and error handling
- Explored techniques for writing maintainable, extensible code

Future Enhancements

- Add more Pokémon types and abilities
- Implement catching wild Pokémon and team management
- Add experience points, levelling, and evolution
- Expand the save system for multiple save files
- Support additional battle modes and multiplayer features

References

- . Official Python documentation
- . Pokémon game mechanics documentation (pokemondb.net, bulbapedia)
- . [Your course textbook/manual]
- . [Any other resources or tutorials used]

THANK YOU!